

FSLog: Adversarial Margin for Cross-System Few-Shot Log Anomaly Detection

Jiyu Tian^{ID}, Mingchu Li^{ID}, and Jianyuan Gan

Abstract—Log-based anomaly detection (LAD) is imperative to ensure both the reliability and security of software systems. Although many deep learning approaches have been designed to capture complex and diverse anomaly patterns from log files, they heavily rely on large-scale annotated data. However, collecting sufficient labeled data is impractical when a software system has just been deployed. In this paper, we propose a cross-system few-shot learning log-based anomaly detection approach, namely FSLog, to solve the abnormal label scarcity problem, which is the main challenge of recent LAD research. Specifically, we leverage a pre-trained model from source system to enrich feature representations so that data instances from target system can also be effectively represented. To this end, we introduce a novel adversarial margin loss to enhance our feature distinguishability while preserving their generalizability. Further, we also develop a masked interactive temporal network for robust feature extraction of temporal relationships for log samples. We evaluate the proposed FSLog on three publicly available datasets based on a standard few-shot learning setup protocol. Experimental results demonstrate that our method achieves the best performance in detecting abnormal logs when compared to state-of-the-art methods.

Index Terms—Log anomaly detection, few-shot learning, fine-tuning, Softmax, margin.

I. INTRODUCTION

LOG data is a valuable source for security analysis, fault diagnosis, and troubleshooting. To reduce system downtime and improve system reliability, log anomaly detection, enabling rapid problem localization, abnormal behavior identification, and attack tracking, has been an active research topic [1], [2], [3]. Recently, various deep learning-based methods are proposed for this log analysis task. They include unsupervised approaches, e.g., DeepLog [4] and LogAnomaly [5], supervised approaches, e.g., LogRobust [6] and LightLog [7], semi-supervised approaches, e.g., PLELog [8], transfer learning-based approaches, e.g., LogTransfer [9].

Received 24 January 2025; revised 28 October 2025; accepted 14 December 2025. Date of publication 2 January 2026; date of current version 12 May 2026. This work was supported in part by the National Nature Science Foundation of China under Grant 625B2031, Grant 62466025 and Grant T2350710232, and in part by the Fundamental Research Funds for the Central Universities under Grant DUT20GJ205. (Corresponding author: Mingchu Li.)

Jiyu Tian and Jianyuan Gan are with the School of Software Technology, Dalian University of Technology, Dalian 116024, China (e-mail: tianjiyu@mail.dlut.edu.cn).

Mingchu Li is with the School of Artificial Intelligence, Jiangxi Normal University, Nanchang 330022, China, and also with the School of Software Technology, Dalian University of Technology, Dalian 116024, China (e-mail: mingchu@jxnu.edu.cn).

Digital Object Identifier 10.1109/TDSC.2025.3647445

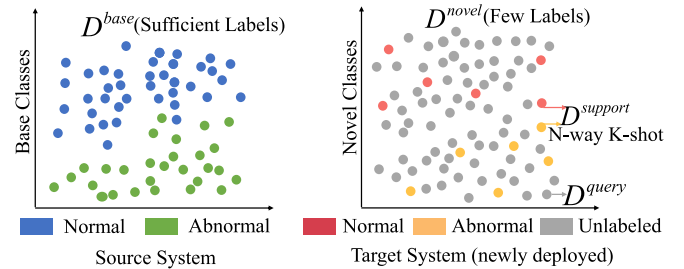


Fig. 1. An illustration of the division of datasets in few-shot learning.

Despite its valuable applications, detecting anomalous logs is still a challenging task. Since there are various software systems (e.g., operating system CentOS, database system SQL Server, and distributed file system HDFS), it is impractical to establish a unified log anomaly detection model that can be applied to all software systems. While re-training a new log anomaly model in a new software system (i.e. target system) is also difficult because labeled logs in the target system are rare and may present different patterns when compared to the log data in the original software system (i.e. source system). We illustrate this research gap in Fig. 1.

Existing log anomaly detection models have not fulfilled the performance requirement due to the following reasons: supervised learning methods for log anomaly detection heavily rely on large-scale labeled training data, which is expensive to obtain and requires tedious manual work. Thus, it is difficult to apply them to any new systems or dynamically evolved systems; unsupervised and semi-supervised learning methods alleviate the labeling problem. However, they are extremely susceptible to log parsing errors since they are unable to adequately learn explicit anomaly features and perform poorly in terms of detection accuracy [10]; transfer learning methods leverage rich prior knowledge from the source system and fine-tune with a few supervisions from the target system to achieve cross-system detection. However, insufficient supervised samples make these methods vulnerable to overfitting. Few-shot learning provides a novel solution for LAD in label-scarce scenarios. However, the detection performance of existing few-shot detection methods is still limited either by the generalization and distinguishability of the model or by the robustness of noisy logs. The research challenges we address in this paper are formalized as:

- **Generalization and Discrimination:** are conflicting objectives in model training. In our study, we found that existing methods only focus on either of them, thus compromising

their overall performance. For instance, LogTransfer [9] focuses on the syntax and noise problems of cross-system logs and only shares the output layer during the fine-tuning stage, while MetaLog [2] pays more attention to global semantic consistency and generalization, ignoring the discriminability of representations. If an effective fine-tuning strategy cannot be designed, the model is likely to over-rely on the prior knowledge of the source system, which leads to a degradation of the model's performance.

- *Robustness*: Existing methods are difficult to process the inherent temporal dependencies and noise in log time sequences. For example, MetaLog [2] uses a GRU network with an attention mechanism to extract features without considering the effect of noise, while Log-MatchNet [11] combines log timestamps with the Bert model, aiming to capture temporal dependencies and semantic relationships, but the Transformer in Bert is more sensitive to noise. Log noise may not only mask useful features but also mislead the model to learn erroneous patterns, thus weakening the robustness of detection.

To address the challenges above, in this paper, we propose a novel few-shot learning-based log anomaly detection method, which aims to improve the transferability of novel classes by fuzzing the decision boundary with an adversarial margin. As shown in Fig. 1, following the fine-tuning-based few-shot learning framework, we establish an explicit support set for the target system based on the "N-way K-shot" principle in meta-learning [12], which serves as the training set for the fine-tuning process. Specifically, (i) first, we innovatively create an adversarial margin cosine softmax (AMCS) loss function for both training and fine-tuning phases, i.e., using cosine softmax with an adaptive rescaling factor to update the feature extractor during pre-training to ensure representational discriminability, and utilizing appropriate negative margins during fine-tuning to ensure generalization to the novel samples; (ii) furthermore, we design a mask-based interactive temporal convolutional network (MITNet) for extracting temporal features of log samples while effectively mitigating noise. Compared to the state-of-the-art (SOTA) log anomaly detection methods [4], [5], [6], [13], [8], [9], [2], [11] proposed in recent years, our approach achieves better performance on classification and generalizability in the presence of scarce annotated samples.

The main contributions of this paper are summarized as follows:

- We proposed a few-shot log anomaly detection method based on the fine-tuning framework for overcoming the reliance of existing methods on large-scale supervision. With sufficient prior knowledge of the base classes from the source system, the novel classes in the target system can be classified using a limited number of samples.
- We designed a novel adversarial margin cosine softmax loss function that balances the transferability and distinguishability of log samples during both the training and fine-tuning phases.
- We introduce a masked interactive temporal network as a backbone for extracting temporal and semantic

relationships of log sequence samples while ensuring the robustness of the representations.

- We provided a method to construct few-shot datasets for base training and fine-tuning corresponding to log anomaly detection based on HDFS, BGL, and Thunderbird datasets and conducted extensive experiments on four few-shot datasets to demonstrate the usability and superiority of our model.

The remainder of the paper is organized as follows: Section II discusses related works. Section III describes the proposed approach. Section IV presents the designed adversarial margin cosine softmax loss. Section V reports the evaluation results of the proposed approach and compares it with advanced approaches. Section VI discusses the advantages and limitations of our approach. Finally, the work is concluded in Section VII.

II. RELATED WORKS

A. Log-Based Anomaly Detection

Existing approaches that use log data for anomaly detection can be grouped into two broad categories: traditional machine learning-based approaches [14], [15] and deep learning-based approaches [16], [17], [18], [9], [13], [19], [11]. Classical machine learning-based approaches require hand-crafted features and a combination of expert knowledge to classify anomalous events in the logs. These methods, such as the PCA-based method [15] and invariant mining (IM) based method [14], required a lot of manual effort in the modeling process and have limited performance in detecting anomalies.

Deep learning-based models have been dominant in log anomaly detection. DeepLog [4] first utilized One-Hot encoding to vectorize the templates during log representation and converts the log sequences into a sparse matrix that contains no semantic information. Noting the semantic information hidden in the log templates, LogAnomaly [5] then modeled log detection as a natural language sequence prediction problem by introducing quantitative vectors and the word2vec algorithm. The LogRobust [6] combined TF-IDF with FastText to counteract evolutionary and parsing noise and introduced an attention mechanism in Bi-LSTM to capture contextual information in log sequences based on the supervised learning framework. LightLog [7] significantly reduced floating-point computation and detection time by introducing semantic vector dimensionality reduction and designing a lightweight TCN network. PLELog [8], which is based on semi-supervised learning, incorporated knowledge of historical anomalies through probabilistic label estimation and designed an attention-based GRU neural network to detect anomalies. LogTransfer [9] used the similarity of logs between different systems and applied transfer learning to transfer the knowledge of anomalies from one software system to another. MetaLog [2] combines globally consistent semantic embedding modules and meta-learning methods to improve the generalization ability of cross-system anomaly detection models. Log-MatchNet [11] extracts the features of log vectors using Bert's model and achieves anomaly detection by calculating the distances between test samples and positive and negative samples through a matching network. Le et al. [10] presented a

unified parsing and grouping of logs, as well as a fair comparison of sophisticated log anomaly detection methods (including DeepLog, LogAnomaly, LogRobust, PLELog, and CNN [20]) on the same platform, pointing out that existing methods still face challenges such as limited labeled data, early detection, evolving systems, and relations among log events.

Recent research has made progress in addressing the challenges of early detection with limited computing power, as described in [7]. The out-of-vocabulary (OOV) problem caused by log evolution as well as parsing noise has also been tackled in the research of [21] and [6] respectively. Experimental results presented in [10] demonstrated that the accuracy of unsupervised and semi-supervised models is difficult to guarantee, especially in the presence of parsing noise. The transfer methods neglected the issue of overfitting during cross-system fine-tuning, which still leads to subpar detection performance. However, given the diversity of log types and the significant differences between software systems, initiating detection projects in scenarios with extremely limited labeled data and log noise remains an urgent problem.

B. Few-Shot Learning

Few-shot learning refers to the training of a model using a limited number of data to achieve good generalization performance on new tasks or novel classes [22]. Existing few-shot learning algorithms can be roughly categorized as either: fine-tuning [23], [24], meta-learning [25], [26], and metric learning [27], [28], [12].

Fine-tuning methods follow the standard transfer learning procedure, which consists of pre-training with base classes and fine-tuning with novel classes. The baseline algorithm and baseline++ algorithm [23], which are widely used for performance comparison, are both implemented based on fine-tuning. The primary distinction between these two algorithms resided in their loss functions, with the baseline algorithm employing the standard softmax and the baseline++ algorithm utilizing the cosine softmax. Meta-learning methods perform a meta-training on multiple subtasks constructed from the base classes during the training phase to enable the learned model can quickly adapt novel tasks during the test phase. MAML [25] is a typical representative of meta-learning in few-shot learning, which does not rely on any specific model architecture. By training a model on multiple related tasks, it can quickly adapt to new tasks with minimal data. Metric-learning methods directly compare the similarities between the query samples and support classes through one single feed-forward pass. Xie et al. [28] proposed a deep brownian distance covariance (DeepBDC) method, which measures the difference between the joint characteristic functions of embedded features and the product of their marginals to learn image representations for few-shot classification.

Few-shot based log anomaly detection methods are a relatively novel subject. To the best of our knowledge, there are almost no few-shot related studies available for comparison on current log anomaly detection methods, and there are no suitable datasets for the benchmark. Introducing few-shot learning is a pioneering and innovative solution in the situation where it is

difficult to generalize existing supervised log anomaly detection approaches.

C. Margin-Based Loss Function

Margin plays a critical role for achieving adequate discrimination between feature representations, and the margin-based softmax loss function has been widely studied [29], [30], [31].

Wen et al. [32] proposed to minimize intra-class variation as much as possible while ensuring that different class features are distinguishable. To encourage intra-class compactness and inter-class separability, Liu et al. [33] proposed a generalized large-margin softmax (L-Softmax) to adjust the decision boundaries. In their further research [34], they proposed to impose discriminative constraints on the hypersphere manifold, that is, to restrict the angular margin (A-Softmax). Deng et al. [29] proposed additive angular margin loss (ArcFace) to further solve the problem of unstable training and difficult convergence in A-Softmax, and significantly improve the discrimination power. Wang et al. [35] reformulated the softmax loss as a cosine loss by normalizing both features and weights to remove radial variations and proposed a large-margin cosine loss (LMCL) to maximize the decision margin in the angular space.

Contrary to the common practice in previous studies, Liu et al. [30] found that the positive margin would expand the intra-class variance of the novel class. Accordingly, the appropriate negative margin would better achieve discriminability and transferability of the model in few-shot learning (Negcos). However, simply introducing negative margins in the pre-training phase ignores the validity of the feature representation. This research inspired us to rethink the discriminability and representativeness of features in few-shot learning.

III. METHODOLOGY

A. Problem Definition

Conventional log anomaly detection approaches typically focus on using a binary classifier to determine labels for log samples. When the number of annotated samples is small, the traditional useful models applied to the problem will encounter overfitting. In the above scenario, learning deep representations driven on base logs and fine-tuning the classifiers with a few support logs of novel classes is an effective research line.

For a few-shot log anomaly detection task, the dataset consists of a base set D^{base} with sufficient labeled logs from the source system and a novel set D^{novel} containing only a few supervisions from the target system. In a task, base samples (source-system samples) can form a base set B , which can be expressed as (B_i, Y_i^B) , where $B_i \in \mathbb{R}^d$, $i = \{1, 2, \dots, I\}$, $Y_i^B = \{1, 2, \dots, Y\}$. The novel samples (target-system samples) can constitute a support set S and a query set Q . Given N classes, each of which has K samples of the same type (N-way K-shot), S can be further denoted as $(S_{j,k}, Y_{j,k}^S)$, where $S_{j,k} \in \mathbb{R}^d$, $j = \{1, 2, \dots, N\}$, $k = \{1, 2, \dots, K\}$, $Y_{j,k}^S$ is the corresponding label. The samples with labels to be verified constitute a query set. The main objective of few-shot log anomaly detection is to accomplish the binary classification task by learning D^{base} and D^{novel} in steps to

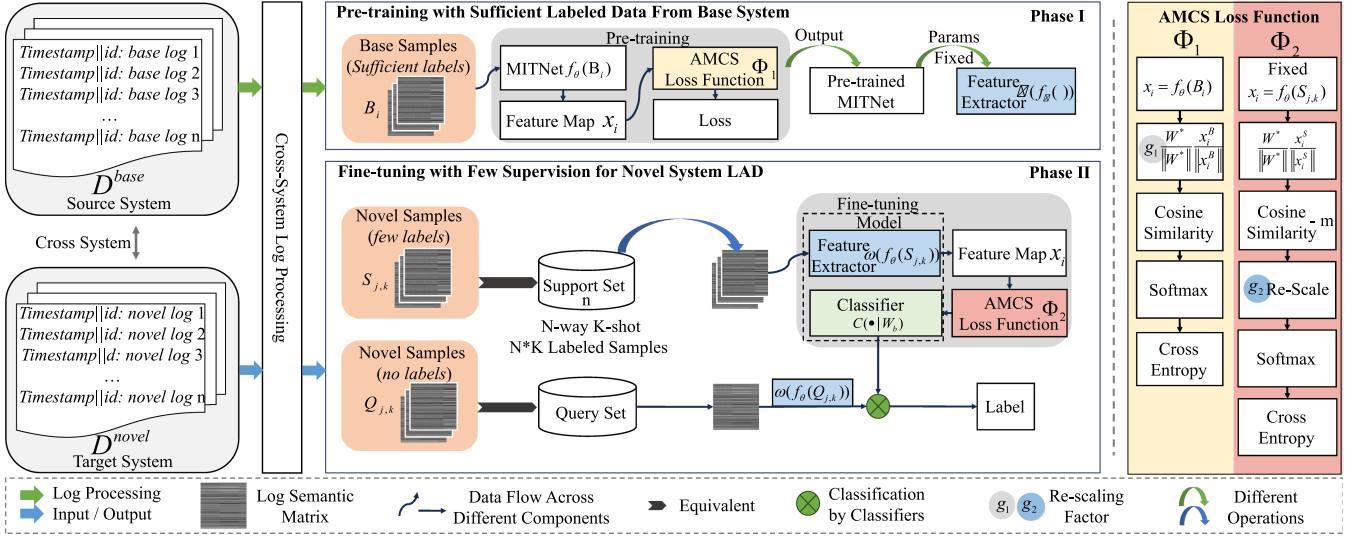


Fig. 2. Overview of FSLog approach, which consists of two phases: the pre-training phase to learn a feature extractor $f_\theta(\cdot)$ with sufficient labeled logs sequences on base classes from source system, and the fine-tuning phase to learn a new classifier $C(\cdot|W_b)$ on novel classes with few labeled log samples from target system.

obtain the ultimate detection model. Firstly, the pre-training phase trains a detection backbone $f_\theta(\cdot)$ (using MITNet as a feature extractor, parametrized by the network parameters θ) with the logs in D^{base} and fixes its weights $f'_\theta(\cdot)$ after training to construct the feature representation x_i . Here, ω represents a fixed parameter operation. Formally,

$$f'_\theta(\cdot) = \omega(f_\theta(D^{base})) = \omega(f_\theta(B_i), Y_i^B) \quad (1)$$

The classifier $C(\cdot|W_b)$ is then fine-tuned with a few labeled novel samples (support set S) to distinguish the unlabeled instances (Query set Q) from novel classes, updating the weights W_b by

$$\begin{aligned} C(\cdot|W_b) &= C(W_b^T \cdot \omega(f_\theta(D^{support}))) \\ &= C(W_b^T \cdot \omega(f_\theta(S_{j,k})), Y_{j,k}^S) \end{aligned} \quad (2)$$

For the general log anomaly detection task, the target of the classification results is expected to match the novel log instance, by minimizing the following loss

$$\arg \min \mathcal{L}^{ce}(\sigma(W_b^T, \omega(f_\theta(S_{j,k}))), Y_{j,k}^S) \quad (3)$$

Where σ is used to obtain the similarity between the two inputs, $\mathcal{L}^{ce}$ denotes the cross-entropy loss between predictions and truth labels.

B. Cross-System Few-Shot Log Anomaly Detection Framework

The proposed FSLog aims to achieve cross-system log anomaly detection by utilizing an effective feature extractor trained with sufficient labeled data from the source system and constructing a classification model with limited supervision from the target system. As illustrated in Fig. 2, following the standard transfer learning paradigm, the FSLog detection model consists of two phases: pre-training and fine-tuning. In the pre-training phase, we utilize sufficiently labeled base class samples

from the source system to train a MITNet model capable of extracting robust temporal features, and fix the network parameters as a feature extractor for the next phase. In the fine-tuning phase, we obtain the feature representations of the novel samples through the feature extractor and train the classifier using a limited number of labeled samples from the target system, while introducing adversarial margin to adjust the intra-class and inter-class distances to eliminate the heterogeneity between different log sets.

Pre-training with Sufficient Labeled Data From Source System: The pre-training phase aims to train an effective backbone $f_\theta(\cdot)$ with abundant annotated log sequences from the base classes. Logs are semi-structured semantic data in the form of text, accompanied by noise and evolution, and require specific processing. Specifically, the original log messages are first parsed into structured templates, then sequenced based on session, sliding, and fixed grouping algorithms. Finally, the templates are mapped into semantic vectors to form a log semantic matrix B_i containing semantic information as input to the feature extractor. It is worth noting that different types of logs usually differ in syntax. Therefore, for cross-system knowledge transfer, it is important to evaluate the similarity of logs between the source system and the target system. LogTransfer provides a cross-system log processing method that not only extracts local contextual information but also takes into account the global word co-occurrence information of template sets, thus reliably measuring the similarity of cross-system logs. (As this is not the focus of this paper, details are provided in [9].) Inspired by [7], the fully convolutional network MITNet $f_\theta(\cdot)$ is able to extract temporal features more quickly and efficiently, compensating for the slow computation of LSTM and the inability of standard CNN to capture causality in time-series data (details are provided in Section III.C). During pre-training, first of all, the samples are input into the backbone to obtain their feature map x_i . Afterwards, the method utilizes the AMCS Φ_1 as a loss function

to gradually update the MITNet weights while extracting more discriminative representations for fine-tuning (more details are provided in Section IV.C).

Fine-tuning with Few Supervision for Target System LAD: With insufficient supervised log samples in the newly deployed system, the few novel classes cannot effectively transform the backbone during fine-tuning [36]. Thus, fine-tuning follows the meta-learning framework, using only a limited number of novel classes to update the classifier parameters. Similar to the pre-training process, the raw logs in the target system are also parsed, grouped, represented, and finally transformed into log semantic matrices based on cross-system log data processing. During fine-tuning, the samples from the novel classes are divided into two sets: the support set S and the query set Q . The support set consists of a few labeled samples from the novel classes, which are used to update the classifier parameters. The query set consists of unlabeled test samples, which are used to evaluate the performance of the updated model. Exactly, in the LAD field, the classification task only involves two classes: normal and abnormal. Therefore, we set N to 2, meaning that K samples are selected from each of the normal and abnormal classes to form a support set with $2 \times K$ samples. Then, the support set is fed into the feature extractor with fixed parameters as the training samples for the fine-tuning phase. The AMCS Φ_2 Loss Function is introduced to fuzz the inter-class variance, which significantly improves the generalization performance of the classifier (details are provided in Section IV.C). Eventually, we use the fine-tuned classifier to detect the query samples and obtain the exact labels.

C. Feature Extraction Backbone MITNet

It has been widely demonstrated that temporal relations, semantic relations, and parsing noise between logs. Therefore, we develop the MITNet feature extractor for log data and introduce it into a fine-tuning-based few-shot learning framework to extract robust temporal semantic representations. As shown in Fig. 3, MITNet consists of four main parts: patch-positional embedding, time masking, dilated convolution, and interactive convolution. Among them, the patch-positional embedding and interactive convolution blocks are a continuation of TSLANet [37], while the event masking and dilated convolution [38] are enhancement modules that we introduce for log data. The specific implementation of MITNet is as follows:

Patch-Positional Embedding: To preserve the temporal dependencies between log events, we employ a positional embedding method. Given an input log time series sample, we first divide it into M patches of length $|p|$ ($\lfloor \cdot \rfloor$ is rounded down) using convolution and map each patch to another embedding vector. Then, the positional embedding is summed with each patch to preserve its temporal order.

Event Masking: first, unlike the continuous values of a standard time series, log samples are discrete values composed of log events in chronological order. Second, log noise is mainly caused by log incorrect parsing, so denoising methods for frequency components become infeasible. Inspired by [39], we adopt a randomized masking patch approach to generate robust features.

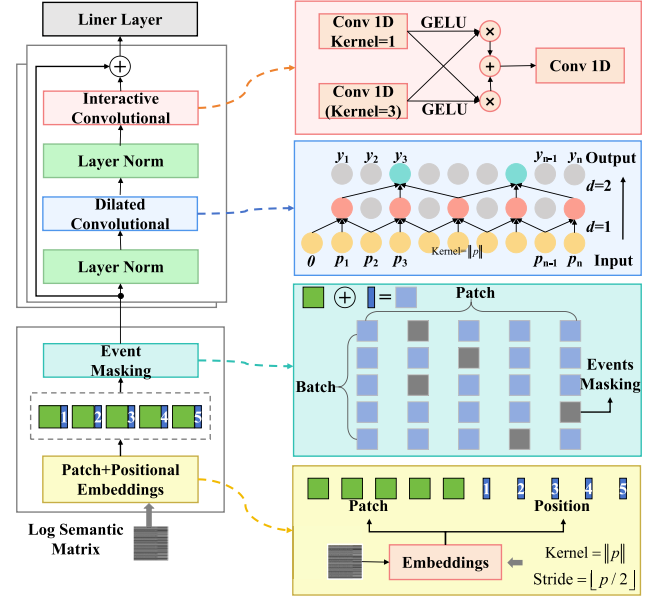


Fig. 3. The MITNet architecture contains four key components: patch-positional embedding, event masking, dilation convolution, and interaction convolution. For the log semantic matrix consisting of log event semantic vectors in temporal order, MITNet uses the following strategies for processing: patch-positional embedding is used to keep temporal ordering; event masking is used to ensure robustness of the representation; dilated convolution is used to expand the receptive field; and interaction convolution is used to extract and fuse features from different scales. After feature extraction, MITNet flattens the 300×300 log sequence into $1 \times n$ -dimensional features for input to the downstream classifier.

Specifically, we randomly mask the hidden vectors after patch and position embedding along the time axis with a binary mask. Since each patch covers multiple log events, masking the patches is equivalent to removing some of the information, which motivates the feature extractor to ensure robust feature extraction in the face of missing information (e.g., incorrect parsing).

Dilated Convolution: To obtain long and effective memories, standard CNN typically require a deep network and a large filter. However, dilated convolution offers an alternative approach that involves the inclusion of a dilation rate d , i.e., interval sampling of the input during convolution. As the depth of the network increases, the receptive field size expands exponentially. Thus, as shown in the figure, the 2-layer dilation convolution that we introduce enables MITNet to obtain long temporal dependencies.

Interactive convolution: consists of two parallel 1D convolutional layers that use different-sized kernels to interact with local features and remote dependencies. Specifically, the first convolutional layer uses a smaller convolutional kernel (e.g., Kernel=1) to capture fine-grained local patterns in the data. The second convolutional layer, on the other hand, uses a larger convolutional kernel (e.g., Kernel=3) to recognize long-duration dependencies. These two features are activated by GELU, fused by multiplication and addition operations, and fed into additional convolutional layers for feature extraction and interactive fusion across different scales.

Overall, compared with the following TSLANet [37] approach, MITNet addresses the problem of log discrete events

and parsing noise by adopting a random event masking strategy to simulate parsing anomalous scenarios, forcing the model to learn robust feature representations, effectively solving the interference of log noise; second, MITNet introduces a double-layer dilated convolution that exponentially expands the filter field with lower computation to capturing long term dependencies. These two improvements enhance the performance in terms of robustness and model efficiency, respectively.

IV. ADVERSARIAL MARGIN COSINE SOFTMAX LOSS

In this section, we begin with a review and standardization of existing softmax-based loss functions. Then, we analyze the differences between the proposed function and existing functions. Finally, we elaborate on the proposed loss function AMCS.

A. Softmax Loss Function Revisited

We denote i -th input samples as z_i and its corresponding label as y_i . Combined with feature extractor $f_\theta(\cdot)$, the feature representations of deep networks can be indicated as $x_i = f_\theta(z_i)$, $x_i \in \mathbb{R}^d$. Based on the problem definition in Section III-A, we provided a unified and standardized explanation of Softmax, LMCL [35], and Negcos [30].

Softmax The most widely used classification loss function, standard softmax loss, can be formulated as follows:

$$L_i = -\log \left(\frac{e^{\|W_{b,y_i}^T\| \|x_i\| \cos(\theta_{y_i})}}{\sum_j e^{\|W_{b,j}^T\| \|x_i\| \cos(\theta_j)}} \right) \quad (4)$$

where weight vector $W_{y_i}^T$ is the y_i -th column of W , b is a bias term (for simplicity, it will be omitted next). The inner product between W_j^T and x_i can be expressed as $\|W_j^T\| \|x_i\| \cos(\theta_{y_i})$, where θ_j is the angle between the vector W_j^T and x_i .

LMCL: However, the standard softmax loss function does not explicitly encourage intra-class compactness and inter-class separability, which results in degeneration for classification performance under large intra-class variations. LMCL is proposed to increase intra-class similarity and inter-class diversity. To eliminate radial variation, the method performs L_2 -normalization of the feature and weight vectors. Additionally, a decision margin is generated by introducing a positive number m ($m > 0$), which makes the classification more rigorous. Since replacing the inner product by the cosine similarity between two feature vectors leads to better performance, this means that the norm of $\|x_i\|$ does not contribute to the scoring function, the method fixes the normalized $\|W_j^T\|$ to 1 and redefines $\|x_i\|$ to s , respectively. Therefore, the altered loss function reinforces the discrimination of learned features by encouraging an extra margin in the decision. Formally, the general formulation of LMCL is defined as

$$L_i = -\log \left(\frac{e^{s \cdot (\cos(\theta_{y_i}) - m)}}{e^{s \cdot (\cos(\theta_{y_i}) - m)} + \sum_{j \neq y_i} e^{s \cdot \cos(\theta_j)}} \right) \quad (5)$$

Negcos: Nevertheless, when LMCL is used for few-shot classifications, adding a positive margin instead can hurt the

performance, while adding an appropriate negative margin could achieve a better tradeoff between the discriminability and transferability. Hence, combining LMCL, the Negcos breaks with the inherent understanding that the margin can only be restricted to a positive integer by redefining the softmax loss with margin ($m < 0$) as follows:

$$\varphi_1 = \beta \cdot (\sigma(x_i, W_{y_i}^T) - m) \quad (6)$$

$$L_i = -\log \left(\frac{e^{\varphi_1}}{e^{\varphi_1} + \sum_{j=1, j \neq y_i} e^{\beta \cdot \sigma(x_i, W_j^T)}} \right) \quad (7)$$

Where σ denotes the similarity function between two input vectors and can be either the inner product or the cosine. β denotes the temperature parameter [40] which could help to apply the appropriate concentrated level around the largest scores and avoids models falling into local optimum solutions.

B. Analysis of AMCS

Few-shot learning can solve the problem of supervision deficits, but the implicit contradiction between the base class and the novel class is ignored, i.e., there is feature heterogeneity between different log sets. Specifically, there are not only similarities but also distributional differences between the base log samples of the source system and the novel log samples of the target system. Similarities between the base and novel log samples may exist due to the presence of common system components, standardized APIs, and shared libraries. In contrast, changes in system functionality, configurations, and user interaction behaviors lead to differences in log sequence patterns between base and novel samples. Therefore, the model trained on the base classes gains outstanding feature discrimination ability and instead fails to fit the novel classes well.

As mentioned in Negcos, when implementing few-shot learning using LMCL, the intra-class variance on the base class decreases and the model becomes more distinguishable on the base class as the positive margins increase during training. However, the dissimilarity between the base class and the new class of samples instead leads to an increase in the intra-class variance of the novel class. Based on the log samples from different systems, we also verified this phenomenon. This is because as the positive margins increase, the model focuses more on correctly classifying existing categories, but lacks sufficient learning and generalization ability for novel categories. In addition, the increase in positive margins can also lead to overfitting of the model, further affecting the classification performance of novel categories. Therefore, as shown in Fig. 4, the ideology of Negcos is to weaken the distinguishability of the model on the base class samples by adding appropriate negative margins in the pre-training phase to avoid overfitting for better transferability. However, using a negative margin in the training process does not achieve separability between classes of

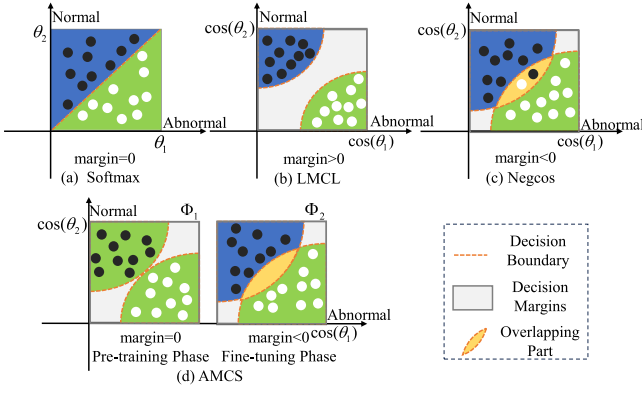


Fig. 4. Decision boundaries of different loss functions under binary classification. (a) $\text{margin}=0$ (e.g., standard Softmax) lacks generalization in few-shot scenarios; (b) $\text{margin}>0$ (e.g., LMCL) increases inter-class separation but may cause overfitting; (c) $\text{margin}<0$ (e.g., NegCos) applies a negative margin during pretraining, which weakens feature discriminability for base model; (d) the proposed method uses $\text{margin}=0$ during pretraining (left) to learn clear and discriminative boundaries from source system data, and introduces a small negative margin ($\text{margin}<0$) during fine-tuning (right) to blur boundaries and improve generalization to novel classes in the target system.

samples and obtain distinguishable feature representations [35]. Consequently, Negcos faces challenges in effectively learning prior knowledge from the base class. Log sequence samples exhibit complex temporal and semantic relationships with one another, making it difficult for feature extractors with limited distinguishability to capture anomalous log patterns accurately.

To resolve the contradiction between generalization and distinguishability in Negcos, our approach focuses on achieving distinguishable but not overfitted feature extractors using sufficient base class samples in the pre-training phase and generalizing the classifier using limited labels from novel classes in the fine-tuning phase. Exactly, by following this approach, the feature extractor learns sufficient prior knowledge from the base class while using the classifier to combat the distributional differences between the new class and the base class. As shown in Fig. 4, different from the single-stage softmax-based loss function, we set the margin to 0 in the pre-training phase and set the margin to negative in the fine-tuning phase. In the pre-training phase, since a positive margin can overfit the model and a negative margin does not guarantee the effectiveness of feature extraction, we do not manipulate the margin to ensure that the extractor achieves distinguishability without hindrance. In the fine-tuning phase, we fix the parameters of the feature extractor and use negative margins to fuzzy the boundaries of the classifier to make the model more transferable.

Theoretically, in the pre-training stage of the source domain, minimizing the softmax cross-entropy loss function is equivalent to accomplishing the dual optimization task described above, expanding both the inter-class distance and compressing the intra-class distribution. Thus, according to Fisher principle, the loss function of the first stage trains a representation model on the source domain that satisfies the following condition, i.e., $\max_{f_\theta(\cdot)} (S_B/S_W)$, where S_B stands for inter-class variance and S_W for intra-class variance. In the fine-tuning stage of the

target domain, a feature extractor freezing strategy is used and a negative Margin mechanism for label scarcity is introduced. This mechanism relaxes the decision boundary from strict inequality $\cos(\theta_{y_i}) \geq \cos(\theta_j + m)$ to allow the target domain samples and the source domain to generate controlled overlap in the boundary region, thus effectively mitigating the inter-domain distribution gap $\|P_s - P_T\|_H$.

C. Adversarial Margin Cosine Softmax Loss

In this subsection, we give specific AMCS loss function implementation details. In AMCS, we introduce a negative margin in stages to counteract the overfitting of the model, i.e., adjusting the margin is incorporated into the fine-tuning process of the classifier. Since the work we followed (i.e., Negcos) introduces the training strategy of temperature parameters [40] when implementing margin-based few-shot learning, it effectively helps to apply appropriate concentration levels around the maximum score, providing more options for parameter optimization during training. Inspired by Negcos, we further introduce two temperature parameters, g_1 and g_2 , to achieve optimization during model training. For a uniform description, we still use z_i for i -th input log samples and x_i for the corresponding feature representation, where $z_i \in \{B_i, S_{j,k}\}$, the remaining cosine similarity is

$$\begin{aligned} \sigma(W_{y_i}^T, x_i) &= \frac{W_{y_i}^T f_\theta(z_i)}{\|W_{y_i}^T\|_2 \|f_\theta(z_i)\|_2} \\ &= \frac{W_{y_i}^T x_i}{\|W_{y_i}^T\|_2 \|x_i\|_2} = \cos(\theta_{y_i}) \end{aligned} \quad (8)$$

In the training phase, we use cosine softmax with parameter scalar to implement the feature extractor, decoupling the direction and length of the vectors. As described in IV-B, the margin of this stage is fixed to 0 to ensure effective feature extraction and avoid overfitting. Additionally, we add a learnable rescaling factor g_1 [23] as an adaptive temperature parameter for training (this is the first time that g_1 has been introduced in a margin-based loss function). Unlike the fixed temperature parameter in Negcos, our strategy can provide adaptive control based on the smoothness and sharpness of the probability distribution, avoiding falling into local optima and allowing the model to converge faster. The general loss formula in the training phase is defined as

$$\Phi_1 = -\log \left(\frac{e^{g_1 \cdot \cos(\theta_{y_i})}}{\sum_j e^{g_1 \cdot \cos(\theta_{y_j})}} \right) \quad (9)$$

In the fine-tuning phase, we appropriately introduce a negative margin to fuzz the decision boundary of the classifier to make the model better generalized to novel classes. It is worth noting that log samples are time series data consisting of log events. Consequently, anomalous samples usually overlap with normal samples in the feature space, which makes samples that are close to the classification boundary sharing high similarity and indistinguishable from each other. To solve the problem, we use a very small temperature parameter g_2 at the fine-tuning process to make the distribution sharper, giving difficult negative

TABLE I
DESCRIPTION OF LOG DATASETS USED IN THE EXPERIMENTS

Datasets	Parsing	Events	Grouping	Average Length	Symbols	Types	Sequences	Few-shot Set	
								Events	Sequences
HDFS	Public [10]	28	Session	19	HF	Normal	575,059	15	2,000
			Session	68		Abnormal	16,838(2.8%)	25	2,000
BGL	Public [10]	378	Session	68	BS	Normal	37,823	53	2,000
			Session	68		Abnormal	31,429 (45.4%)	109	2,000
			Sliding(size = 300, step=100)	300	BT	Normal	41,457	46	2,000
			Sliding(size = 300, step=100)	300		Abnormal	6,007 (12.7%)	152	2,000
TBird	Spell [41]	6,411	Session	300	TB	Normal	57,969	516	2,000
			Session	300		Abnormal	41,780 (41.9%)	441	2,000

samples a larger gradient to separate from positive samples. As a consequence, the classifier can better capture the heterogeneity between the base class and the novel class and interfere with the features of the novel instances in an adversarial manner. The general loss formula for the fine-tuning phase is defined as

$$\Phi_2 = -\log \left(\frac{e^{\frac{1}{g_2} \cdot (\cos(\theta_{y_i}) - m)}}{e^{\frac{1}{g_2} \cdot (\cos(\theta_{y_i}) - m)} + \sum_{j=1, j \neq y_i} e^{\frac{1}{g_2} \cdot (\cos(\theta_j))}} \right) \quad (10)$$

Combining (9) (10), the formula (11) implements the margin equilibrium for base class transferability and novel class discriminability.

$$L_{i,p} = \begin{cases} \Phi_1, & z_i \in f'_\theta(B) \\ \Phi_2, & z_i \in f'_\theta(S) \end{cases} \quad (11)$$

V. EXPERIMENT

In our study, we evaluate our approach by answering the following research questions.

RQ1: How does the FSLog method perform in terms of effectiveness in label scarcity scenarios?

RQ2: How does the FSLog method perform in terms of advancement?

RQ3: How robust does the FSLog method perform in terms of noise scenarios?

RQ4: How sensitive is the FSLog method to hyperparameters?

RQ5: How does each component contribute to FSLog?

A. Experimental Setup

1) *Datasets:* In the field of log anomaly detection, the widely used datasets include various sources. However, there is no suitable public dataset available for few-shot log anomaly detection. To evaluate the proposed method, we consider the raw log messages contained in HDFS, BGL, and Thunderbird as the data sources, and then reconstruct the few-shot datasets as HDFS-FS-Session (HF), BGL-FS-Session (BS), BGL-FS-Time (BT), and TBird-FS-Time (TB). The details of each raw dataset are as follows:

The *HDFS dataset* [15] consists of 11,175,629 logs produced from more than 200 Amazon EC2 nodes. These log messages form different log samples according to their block ID, including 575,059 normal log sequences and 16,838 abnormal log sequences (2.8%).

TABLE II
RESULTS OF THE EXISTING UNSUPERVISED AND SEMI-SUPERVISED LAD APPROACHES

Test	Methods	PRE	REC	FPR	F1
HF	DeepLog	85.4	99.4	17	91.9
	LogAnomaly	86.5	91.8	14.4	89.1
	PLELog	88.6	98.5	12.7	93.3
	Proposed	92.7	94.3	7.5	93.5
	Percentage Increase	+4.1%	-4.2%	+5.2%	+0.2%
BS	DeepLog	99.6	100	0.4	99.8
	LogAnomaly	99.5	99.8	0.5	99.6
	PLELog	99.6	98.8	0.4	99.2
	Proposed	97.6	97.2	2.4	97.4
	Percentage Increase	-2.0%	-2.8%	-2%	-2.4%
BT	DeepLog	52.5	97.7	88.4	68.3
	LogAnomaly	53.6	98.2	85	69.3
	PLELog	73.3	86.5	31.5	79.4
	Proposed	91.8	91.5	8.1	91.6
	Percentage Increase	+18.5%	+5.0%	+23.4%	+12.2%
TB	DeepLog	56.7	99.7	76.2	72.3
	LogAnomaly	58.9	99.3	69.3	73.9
	PLELog	61.1	98.8	62.9	75.5
	Proposed	80.0	82.2	20.5	81.1
	Percentage Increase	+18.9%	-16.6%	+42.4%	+5.6%

The *BGL Dataset* [42] consists of 4,747,963 logs generated by a supercomputing system Blue-Gene/L. As shown in Table I, we use the session window and sliding window to divide the log sequence respectively and label the log sequence as abnormal if there are any faulty logs in the sequence. When using the session window, we construct 69,252 log sequences based on its node ID, of which 31,429(45.4%) log sequences are labeled as malfunctions. When using the sliding window, we set the size of the sliding window to 300 and the step size to 100, a total of 6,007(12.7%) abnormal sequences were obtained by this method.

The *thunderbird dataset (TBird):* [42] consists of more than 200 million logs collected from a Thunderbird supercomputer at Sandia National Labs (SNL). As presented in Table I, we set the window size to 300 and the step size to 100, use the sliding window to divide the logs, and obtain 41,780 anomalous sequences (41.9%).

The reconstruction of the dataset first requires log processing. The use of Spell [41] to parse the logs was in response to the high susceptibility of the unsupervised learning approach to noise as mentioned in Section II-A. We follow up the work in [6], [7], hence the dimension d of the semantic vectors is 300, and the window size w is also 300. After parsing, grouping, and semantic

representation, the length of each log sequence $d \times w$ is 300×300 (Note that $d=w$ is not a strict requirement, and the introduced MITNet can effectively process input dimensions and unequal input lengths).

Next, according to the specific conditions of different data sources, the four few-shot datasets HF, BS, BT, and TB were constructed. Every few-shot dataset contains normal and abnormal sequences. For each type of set, we chronologically selected 2,000 samples as the data source to construct few-shot query sets for testing. Each detection task was defined as a few-shot binary classification task to distinguish between normal samples and abnormal samples from the same dataset, i.e., N in N -way K -shot is 2.

Experiments are conducted on four few-shot datasets separately. For HF and BS, they contain two types of log sequences that are grouped by session, respectively. We utilize BS as the data source for the base set and the remaining one as the data source for the novel set. Conversely, predictions for BS using HS as the base class also hold. Correspondingly, BT and TB can be used as base sets for each other. Hence, there are four combinations, providing four sets of experiments. A sufficient number of training and testing tasks are constructed for experiments. The model has to learn from the K ($K=5, 20$) samples using chronological selection, performing the real few-shot detection of normal and abnormal log sequences.

2) *Evaluation Metrics*: To measure the effectiveness of models in anomaly detection, we use Precision (PRE), Recall (REC), False Positive Rate (FPR), and F1 score (F1) as detection metrics. Among them, precision is the proportion of all log sequences identified as anomalous that are correctly identified as anomalous. $Precision = \frac{TP}{TP+FP}$ Recall is the proportion of all log sequences that are correctly identified as anomalous. $Recall = \frac{TP}{TP+FN}$ FPR is the proportion of misclassified normal samples to all normal log sequences. $FPR = \frac{FP}{TN+FP}$ F1 score is the summed average of precision and Recall. $F1\ score = \frac{2 * Precision * Recall}{Precision + Recall}$

TP : is the number of abnormal samples correctly detected as abnormal. FN is the number of abnormal samples that are not detected by the model. TN is the number of normal samples that are correctly detected as normal samples. FP is the number of normal samples wrongly identified as abnormal.

3) *Implementations and Environments*: In our experiments, we process the log data and conduct log-based anomaly detection as follows: we adopt the implementations of log parsing provided by open source [10] and log representation based on FastText. We use SGD as the optimizer, set the learning rate to 0.01, the momentum of 0.9 for 100 iterations in fine-tuning, weight_decay to 0.0005, set the number of epochs to 50. Furthermore, the classifiers being compared are all implemented using the scikit-learn library. We set neighbors = 5 for KNN, kernel to linear in SVM, and tuned C in the interval $[0,1]$, and other classifier hyperparameters as default.

To complete the testing of the method, the experiments are performed using the following hardware and software platforms: Intel(R) Xeon(R) Gold 5218 CPU @ 2.30 GHz, CPU cores 16, 256 GB RAM, NVIDIA Tesla V100-PCIE-32 GB*2, CentOS

Linux release 7.9, CUDA 11.7, cuDNN 7.6.5, Python 3.7.12, and PyTorch 1.7.0. All of the conducted experiments took about two months to complete, using two physical GPUs.

B. RQ1: Comparison With Different Log Anomaly Detection Approaches

This section compares the proposed few-shot detection method with the existing unsupervised and semi-supervised baseline methods DeepLog, LogAnomaly and PLELog in Table II, as well as the supervised and transfer learning baseline methods LogTransfer, NeuralLog, LogRobust, LogGroup, Log-MatchNet and MetaLog in Fig. 5. Except for the methods proposed in this paper, none of the other considered approaches are the few-shot detection ones. Please note that to fairly simulate the scarce scenario of anomalous samples, in the conducted experiments, the normal samples of the novel sets available for each unsupervised and semi-supervised learning method are sufficient, while the samples for the supervised and transfer methods adhered to the N -way K -shot limitation and we rotate the training and testing types of samples. Taking Fig. 5 as an example, the first experiment (Test on HF) uses the samples of the BS dataset as base classes for pre-training, and the samples of the HF dataset as novel classes for fine-tuning and testing. Since the feature extractor was trained on a large number of BS samples, the labeled samples in the HF dataset represent the support set and are only used to fine-tune the classifier using a limited number of K ($K=5$ or 20 , i.e., 2-way 5-shot or 2-way 20-shot) samples.

When $K=5$, the FSLog achieves the best accuracy among the five methods, having an averaged F1 of 96.7 percentage points (abbreviated as pp hereafter) on the BS dataset, 89.1 pp on the BT dataset, 76.6 pp on the HF and 77.5 pp on the TB dataset as illustrated in Fig. 5. As shown in Table II, DeepLog, PLELog, and LogAnomaly show significant advantages on the HF and BS datasets and even achieve an F1 of over 99.0 pp on the BS dataset with a false alarm rate of less than 0.5 pp. However, on the other two datasets based on chronological selection, these methods fail to achieve effective detection. This demonstrates that unsupervised and semi-supervised learning methods still cannot be applied in to newly deployed software systems. The detection performance of the metric-based few-shot learning method Log-MatchNet, while higher than supervised and transfer learning methods, is lower than that of FSLog, and even slightly lower than that of meta-learning methods in most scenarios.

When $K=20$, the FSLog has significantly improved, having an averaged F1 of 96.9 pp on the BS dataset, 92.8 pp on the BT dataset, 86.1 pp on the HF dataset, and 80.3 pp on the TB-FS dataset as illustrated in Fig. 5. Even compared to unsupervised and semi-supervised learning models trained on sufficient labels as shown in II, our method still has an absolute advantage. Obviously, the transfer and supervised methods used for comparison still suffer from overfitting. This indicates that existing methods are still powerless for application scenarios such as cross-system cold starts, even with a slight increase in supervision. Moreover, despite outperforming other methods in terms of detection performance, FSLog still reaches a 20%

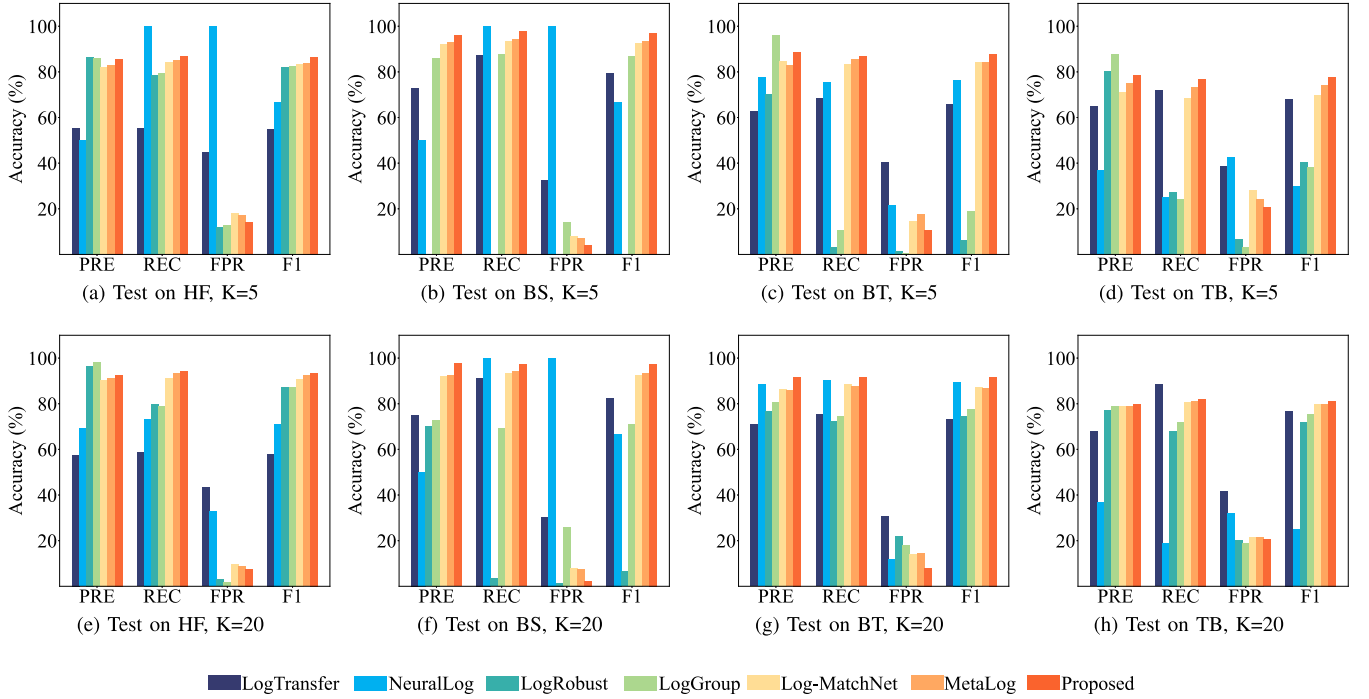


Fig. 5. Comparison with the existing supervised LAD approaches.

TABLE III
DETECTION EFFICIENCY OF EXISTING LAD METHODS

Method	Inference Time (s)	FLOPs (M)	Params (K)
LogTransfer	1.3	6.0	295.1
NeuralLog	3.2	1320.0	32,280
LogRobust	0.89	252.3	835.8
LogGroup	3.3	944.3	6421.0
Log-MatchNet	676.2	27,250	85,290
MetaLog	489.6	145.9	423.9
FSLog	319.4	5.9	105.7

false alarm rate on TB, hindering the online deployment of the detection model. However, our approach is still effective in coping with the cold-start phase where samples are scarce. The proposed few-shot approach provides a feasible solution to this challenge of cross-system detection.

In addition, Table III demonstrates the significant differences between the models in terms of inference time, floating point operations (FLOPs), and number of parameters (Params). In terms of inference time, LogRobust has a clear advantage, being almost 760 times faster than the slowest Log-MatchNet. The supervised inference framework makes LogRobust highly efficient, while the matching network-based inference model leads to high computational consumption and low computational efficiency of Log-MatchNet. Although the inference speed of FSLog is slower than that of methods such as LogRobust, it has the lowest FLOPs (5.9 M) and parametric count (105.7 K), which are only 0.02% and 0.12% of that of Log-MatchNet, and it can be adapted to most of the practical application scenarios. Moreover, it has obvious advantages in detection performance

and is more suitable for the cold start phase of the system with limited labels and less demanding requirements on detection efficiency.

C. RQ2: Comparison With the State-of-The-Art Few-Shot Learning Approaches

Table IV compares FSLog and current few-shot classification methods on four datasets. According to novel classes on HF dataset, for the 5-shot task, FSLog achieved the best detection results on all three evaluation metrics. On the comprehensive evaluation criterion F1, FSLog outperforms RFS, which is also based on fine-tuning, by 6.6 pp, even exceeding the baseline by 21.5 pp. With the increase of labeled samples in the support set, the detection rate of FSLog is further improved. We achieve 86.0/86.3/86.1 pp for 20-shot tasks on each of the three evaluation criteria, outperforming R2D2 85.4/86.0/85.7 pp (based on meta-learning, the second-best result). Several few-shot learning methods based on metric learning have not achieved impressive results.

From BS dataset, our methods are high-ranking across the board. Compared to RENet (based on metric learning), the gains of FSLog are 0.4 pp and 0.2 pp on F1 for the 5-shot/20-shot task, respectively. Using HF as the base set for the prediction of BGL-FS-Session, FSLog outperforms even a detection model trained with sufficient labeled samples. Furthermore, all four fine-tuning-based methods exceed 90 pp. The results for BT dataset are shown in Table IV. For the 5-shot task, FSLog results are the best. But for the 20-shot task, RFS is very competitive and is much better than the other methods except R2D2 and ours. We are slightly inferior to RFS, being only 0.3 pp below

TABLE IV
EXPERIMENT RESULTS OF FEW-SHOT LEARNING APPROACHES ON DIFFERENT FEW-SHOT DATASETS

Test	Test Set	Types	Classifiers	Backbones	K=5				K=20			
					PRE	REC	FPR	F1	PRE	REC	FPR	F1
BS → HF	HDFS-FS-Session	Fine-tuning	Baseline [23]	ResNet18 ²	55.5	54.8	43.9	55.1	58.6	55.1	38.9	56.8
			Baseline++ [23]	ResNet18	66.0	66.4	34.2	66.2	74.4	74.5	25.7	74.4
			RFS [43]	ResNet12 ³	71.2	69.8	28.3	70.0	74.7	72.2	24.5	73.3
			DiffKendall [24]	ResNet12	72.8	75.8	28.3	<u>74.3</u>	89.0	91.9	11.4	<u>90.4</u>
			Proposed MITNet		85.8	87	14.4	86.4	92.7	94.3	7.5	93.5
		Meta-learning	Percentage Increase		+13.0%	+11.2%	+13.9%	+12.1%	+3.7%	+2.4%	+3.9%	+3.1%
			R2D2 [44]	ResNet12	74.1	75.0	26.2	74.5	85.4	86.0	14.7	85.7
			DeepBDC ¹ [28]	ResNet12	51.7	48.8	45.6	50.2	60.4	60.4	39.6	60.3
			ProtoNet [27]	ResNet12	65.6	67.9	35.6	66.7	67.2	70.1	34.2	68.6
			RelationNet [45]	ConvNet ⁴	53.6	55.9	48.4	54.7	59.1	60.0	41.5	59.5
		Metric-learning	CAN [46]	ResNet12	56.3	56.0	43.5	56.1	57.8	57.7	42.1	57.7
			RENet [47]	ResNet12	59.5	59.3	40.4	59.4	61.6	65.8	41.0	63.6
HF → BS	BGL-FS-Session	Fine-tuning	Baseline [23]	ResNet18	89.6	91.3	10.6	90.4	92.0	90.0	7.8	91.0
			Baseline++ [23]	ResNet18	92.7	92.7	7.3	92.7	95.1	94.8	4.9	95.0
			RFS [43]	ResNet12	90.1	89.4	9.8	89.8	90.9	90.9	9.1	90.9
			DiffKendall [24]	ResNet12	92.2	94.3	7.9	93.2	91.6	94.1	8.7	92.8
			Proposed MITNet		96.1	97.9	3.9	97.0	97.6	97.2	2.4	97.4
		Meta-learning	Percentage Increase		-0.3%	+1.4%	-0.3%	+0.7%	+0.9%	+0.5%	+0.9%	+0.7%
			R2D2 [44]	ResNet12	90.6	90.3	9.4	90.4	94.3	93.9	5.7	94.1
			DeepBDC [28]	ResNet12	84.5	86.6	15.9	85.6	86.1	88.	14.3	87.3
			ProtoNet [27]	ResNet12	89.4	89.2	10.6	89.3	84.3	85.3	15.9	84.8
			RelationNet [45]	ConvNet	68.3	68.5	31.8	68.4	46.6	46.2	52.9	46.3
		Metric-learning	CAN [46]	ResNet12	85.6	85.9	14.5	85.7	95.5	95.4	4.5	95.5
			RENet [47]	ResNet12	96.4	96.5	3.6	<u>96.3</u>	96.7	96.7	3.3	<u>96.7</u>
TB → BT	BGL-FS-Time	Fine-tuning	Baseline [23]	ResNet18	79.8	81.1	20.5	80.4	82.6	82.1	17.3	82.4
			Baseline++ [23]	ResNet18	83.5	83.4	16.5	83.5	85.8	85.5	14.1	85.6
			RFS [43]	ResNet12	84.7	85.4	15.4	<u>85.0</u>	93.0	93.1	7.0	93.1
			DiffKendall [24]	ResNet12	85.1	85.6	15.0	85.3	87.2	89.1	13.1	88.1
			Proposed MITNet		88.8	86.9	10.9	87.8	91.8	91.5	8.1	91.6
		Meta-learning	Percentage Increase		+4.1%	+1.5%	+4.5%	+2.8%	-1.2%	-1.6%	-1.1%	-1.5%
			R2D2 [44]	ResNet12	86.2	87.0	13.9	86.6	92.5	91.9	7.5	92.2
			DeepBDC [28]	ResNet12	85.8	92.5	15.3	89.0	79.2	79.5	20.9	79.3
			ProtoNet [27]	ResNet12	81.5	82.1	18.7	81.8	78.0	76.5	21.6	77.2
			RelationNet [45]	ConvNet	56.0	58.4	45.9	57.1	55.7	59.0	46.9	57.2
		Metric-learning	CAN [46]	ResNet12	71.1	71.4	29.0	71.2	73.8	73.1	25.9	73.4
			RENet [47]	ResNet12	86.1	86.8	14.0	86.4	85.3	86.4	14.9	85.9
BT → TB	TBird-FS-Time	Fine-tuning	Baseline [23]	ResNet18	78.3	78.3	21.7	78.3	85.3	85.5	14.7	85.4
			Baseline++ [23]	ResNet18	75.8	76.9	24.5	76.3	82.9	81.7	16.9	81.8
			RFS [43]	ResNet12	72.5	72.4	27.5	72.4	82.0	82.9	18.2	82.4
			DiffKendall [24]	ResNet12	76.0	71.9	22.7	73.9	77.0	80.1	25.0	78.5
			Proposed MITNet		78.7	76.9	20.8	77.8	80.0	82.2	20.5	81.1
		Meta-learning	Percentage Increase		+0.4%	-1.4%	+0.9%	-0.5%	-5.3%	-3.3%	-5.8%	-4.3%
			R2D2 [44]	ResNet12	63.2	62.1	36.2	62.6	73.3	73.8	26.9	73.6
			DeepBDC [28]	ResNet12	56.0	56.6	44.5	56.2	61.2	60.0	38.1	60.5
			ProtoNet [27]	ResNet12	65.7	65.9	34.4	65.7	70.3	72.3	30.5	71.3
			RelationNet [45]	ConvNet	61.6	63.5	39.6	62.5	73.4	74.7	27.1	74.0
		Metric-learning	CAN [46]	ResNet12	65.5	65.0	34.3	65.3	60.9	62.2	39.9	61.5
			RENet [47]	ResNet12	64.3	65.1	36.1	64.7	68.7	68.6	31.3	67.9

¹ DeepBDC contributed two structures, and here we choose Meta-DeepBDC for comparison.

² this ResNet18 network was designed following the practice of the following papers [48].

³ this ResNet12 network is implemented by following [49].

⁴ this ConvNet is adapted from [27].

it on the F1. From TB dataset, the baseline is superior among our competitors. However, for the 20-shot task, our REC, FPR, and F1 still reached 80.7pp, 0.7pp, and 80.3pp respectively. This does not prevent our method from having the best overall detection performance.

In summary, it can be observed that the fine-tuning-based few-shot learning approaches are more suitable for log anomaly detection, which is also the basis for our improvement direction. These comparisons demonstrate that our models have stronger domain transfer capability.

D. RQ3: Robustness Analysis

To verify the robustness of FSLog to parsing noise, we conduct experiments using different log parsers and zero-vector injection. First, following [10], we employ multiple log parsers (Spell,

AEL, Drain, Brain) to demonstrate FSLog performance under different parsing noise. Additionally, to simulate the worst-case scenario where parsed logs completely lose semantic information, we randomly replace a certain percentage of events in the log sequence samples with zero vectors (which lose sample information to varying degrees) to evaluate the sensitivity of the detection model to missing noise.

As illustrated in Fig. 6, FSLog exhibits extremely high robustness on all datasets. The detection performance of FSLog remains stable in the log data generated by different parsers. In particular, the F1 of FSLog is basically the same for the three log data parsed by Spell, Drain, and Brain. On the log data parsed by AEL, the detection performance of FSLog is slightly lower than the other three parsers, but the performance drop is not significant.

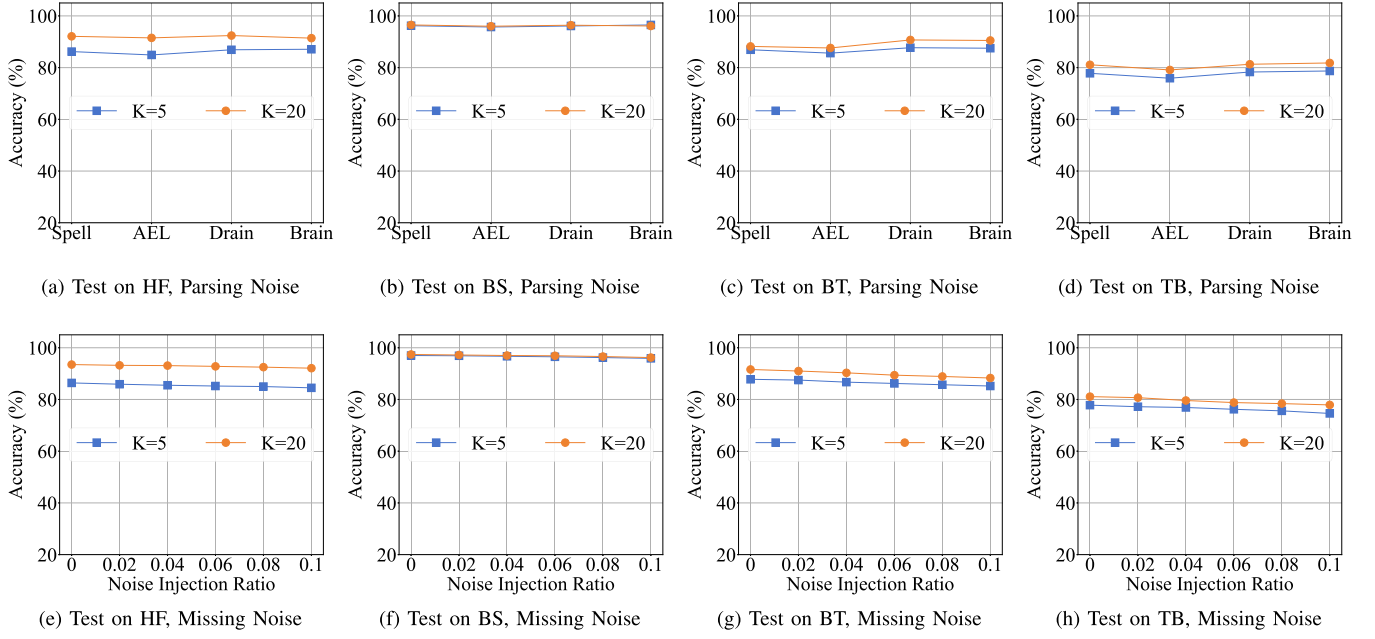


Fig. 6. Impact of Injection Noise and Parsing Noise on Detection Performance.

In the noise-injected scenario, when $K=5$ and the noise ratio is increased from 0% to 10%, the F1 scores of FSLog on the HF, BS, BT, and TB datasets decrease by 1.9 pp, 1.1 pp, 2.6 pp, and 3.2 pp, respectively. When $K=20$ and the noise ratio is similarly increased from 0% to 10%, the F1 score of FSLog decreases by 1.4 pp, 1.2 pp, 3.3 pp, and 3.2 pp. It is worth noting that for the HF and BS datasets, the detection performance of FSLog is almost unaffected by noise. For the BT and TB datasets, although the detection performance slightly decreases when the noise reaches 10%, FSLog still maintains a good detection performance compared to the existing methods shown in Fig. 5, which indicates that FSLog has excellent robustness to deal with log noise effectively.

E. RQ4: Comparison With Different Classifiers and Window Size

In this study, the LAD framework is built upon a fine-tuned implementation of classifiers based on the Multilayer Perceptron (MLP) structure. To demonstrate the effectiveness of the classifier, a comparative analysis is conducted using different classifiers in addition to those employed in this study. The experimental results are elaborated in Fig. 7. For ease of comparison, we use the Baseline++ model that applies ResNet18 as ConvNet to feature extraction and use cosine similarity as the metric. In addition, the size of the sliding window affects the detection performance. To better investigate the effect of window size on FSLog, we use window size as a hyperparameter and test the sensitivity of FSLog under four different window sizes: 300, 100, 50, and 20.

To assess the overall performance of the classifiers, we conduct feature extraction using the MITNet with fixed parameters. The extracted features are then inputted into five different

classifiers, namely KNN, SVM, GaussianNB, and AdaBoost, for anomaly detection. The experimental results depicted in Fig. 7 provide compelling evidence that the classifier when paired with AMCS, attains superior performance. Particularly, when $K=5$, our classifier outperforms the other classifiers by a significant margin on the BT dataset, demonstrating the capability to approach a commendable 90.0pp in the comprehensive evaluation metric F1, while the F1 of KNN, SVM, GaussianNB, and AdaBoost are 75.9 pp, 64.6 pp, 73.3 pp and 75.2 pp respectively, which are more than 15 pp lower than the proposed classifiers. On the other three datasets, our classifier consistently maintains its advantage and achieves an FPR of 3.3 pp on the BS dataset. When $K=20$, a noteworthy improvement in performance is observed across all classifiers.

As shown in Fig. 8, for different window sizes, on the BT dataset, the detection performance of FSLog remains basically the same when w is 300, 100, and 50, respectively. When $w=20$, the F1 of FSLog decreases by 1 pp and 0.8 pp in the scenarios with $K=5$ and $K=20$, respectively. on the TB dataset, the performance of FSLog is similar at $w=300$ and $w=20$, but the performance shows a clear decrease at $w=100$ and $w=50$. In particular, when $K=5$ and $w=50$, the F1 of FSLog decreases by 3 pp compared to $w=300$, which shows that FSLog is sensitive to the time window size. However, compared with the existing methods in Table II and Fig. 5, FSLog maintains leading performance under different windows.

F. RQ5: Ablation Studies

To investigate the effects of core components in FSLog, we conduct extensive ablation studies either in the absence of each component or by replacing them with others and compare the results in the 5-shot and 20-shot settings. In Table V, the

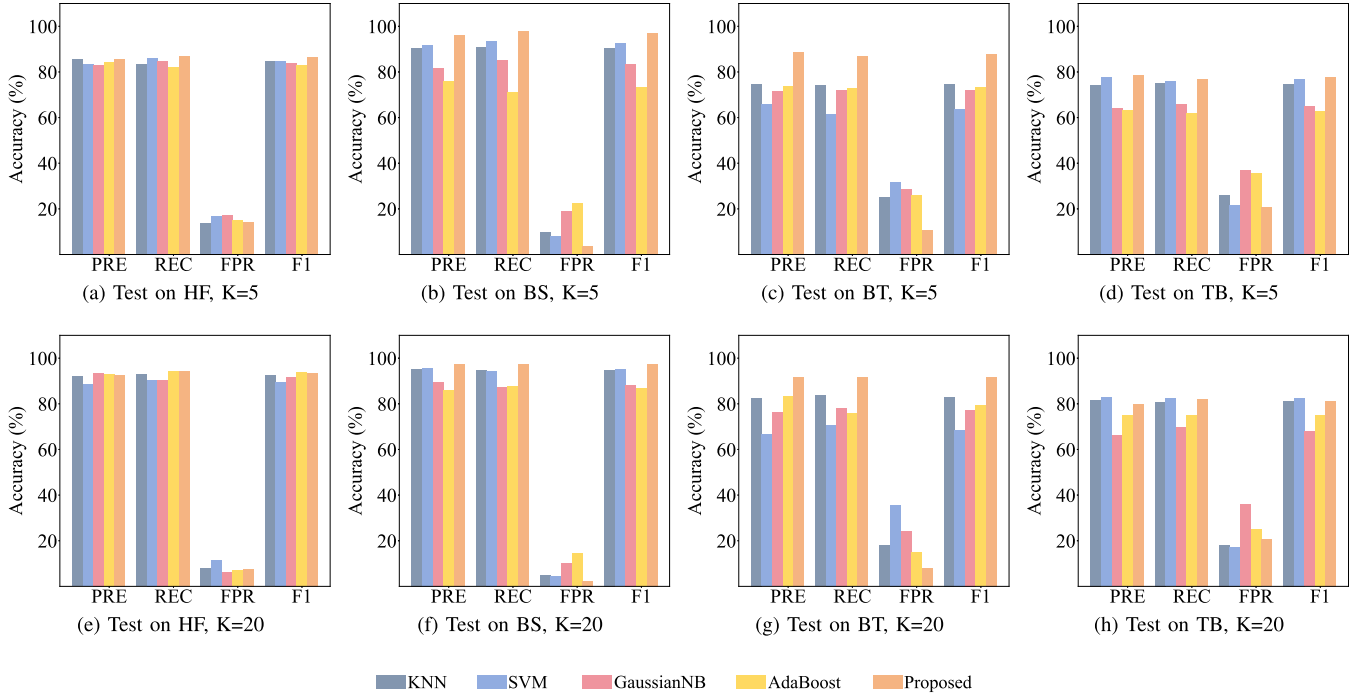


Fig. 7. Comparison with different classifiers.

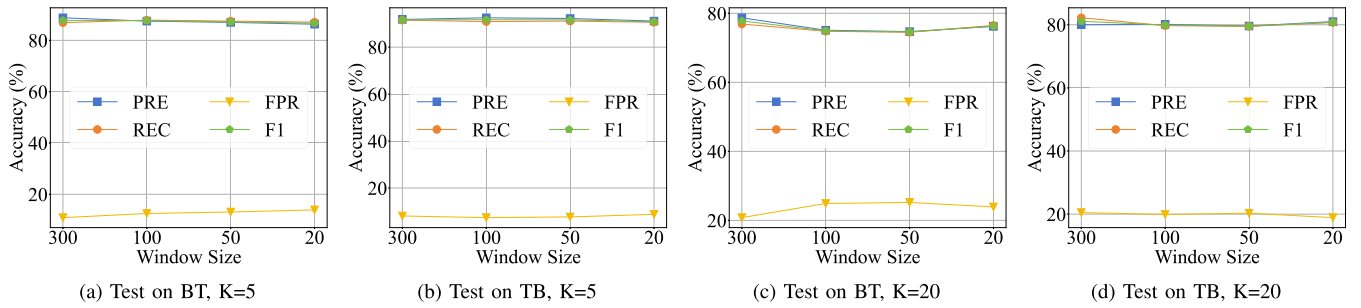


Fig. 8. Comparison with different window size.

$\checkmark$ indicates that each component is utilized in the experiments, while the $\times$ indicates that the component is not employed.

As log sequences contain both temporal and semantic information, we introduce MITNet with dilated and denosing convolutional network to capture the relatedness between logs. As shown in Table V, compared to ResNet18, when using MITNet as the feature extractor, although the detection performance is slightly lower for the TB dataset, there is an improvement on the other three few-shot datasets. Especially on the BS dataset, the highest F1 has achieved 96.8 pp (for 5-shot) and 97.0 pp (for 20-shot).

For each fine-tuning task of FSLog based on the meta-learning framework, we need to build and train a new classifier that introduces new parameters. In Table V, we quantify the classifiers constructed with several different loss functions. The proposed AMCS loss function outperforms all baseline methods (e.g., Cosine, LMCL, Arcface, and Negcos) in the detection accuracy

by a staged margin. However, the TB dataset is not learned by the model using the Negcos loss, and accordingly, the detection results are very poor, even similar to random guessing. This further indicates that only fuzzing the decision boundary of the classifier during fine-tuning is more suitable for heterogeneous and diverse log data.

Notably, in the experimental results in Table V, the combination of AMCS and MITNet achieved the best performance in five out of the eight test scenarios (4 datasets $\times$ K = 5/20). However, in the remaining three scenarios, MITNet+ArcFace and ConvNet+AMCS slightly outperform FSLog, respectively. The possible reasons are: the target domain anomaly samples of the HF dataset contain relatively simple log types, and the classification strategy of ArcFace by increasing the interclass distance may be more adapted to this kind of scenario; AMCS may slightly blur the classification boundary, leading to a slight underperformance of FSLog's detection performance. However,

TABLE V
RESULTS OF ABLATION EXPERIMENTS WITH FSLOG

ConvNet	MITNet	Components					Test	K=5				K=20			
		Cosine	Negcos	LMCL	ArcFace	AMCS		PRE	REC	FPR	F1	PRE	REC	FPR	F1
✓	✗	✓	✗	✗	✗	✗	BS → HF	66.0	66.4	34.2	66.2	74.4	74.5	25.7	74.4
							HF → BS	92.7	92.7	7.3	92.7	95.1	94.8	4.9	95.0
							TB → BT	83.5	83.4	16.5	83.5	85.8	85.5	14.1	85.6
							BT → TB	75.8	76.9	24.5	76.3	82.9	81.7	16.9	81.8
✓	✗	✗	✗	✗	✗	✓	BS → HF	76.7	75.9	23.1	76.3	83.4	82.5	16.5	82.9
							HF → BS	93.6	93.2	6.4	93.4	95.5	95.1	4.5	95.3
							TB → BT	86.6	87.1	13.5	86.8	89.9	89.6	10.1	89.7
							BT → TB	78.5	78.9	21.7	78.7	82.6	83.0	17.5	82.8
✗	✓	✓	✗	✗	✗	✗	BS → HF	76.5	78.5	24.1	77.5	81.5	83.5	18.9	82.5
							HF → BS	95.8	97.2	4.3	96.5	97.7	96.9	2.3	97.3
							TB → BT	83	83.2	17.0	83.1	84.2	84.5	15.9	84.3
							BT → TB	60.9	64	41.1	62.4	77.0	75.5	22.6	76.2
✗	✓	✗	✓	✗	✗	✗	BS → HF	79.2	81.9	21.5	80.5	83.4	86.7	17.3	85.0
							HF → BS	90.3	93.3	10.1	91.8	96.2	95.5	3.7	95.8
							TB → BT	51.9	62.1	57.6	56.5	55.4	58.5	47.1	56.9
							BT → TB	50.6	52.6	51.3	51.6	50.6	52.9	51.5	51.7
✗	✓	✗	✗	✓	✗	✗	BS → HF	79.6	83.7	21.5	81.6	86.9	89.6	13.5	88.2
							HF → BS	90.4	89.9	9.5	90.1	96.6	95.7	3.3	96.1
							TB → BT	80.6	79.5	19.1	80	85.4	85.5	14.7	85.4
							BT → TB	59.4	59.2	40.5	59.3	65.2	69.2	36.9	67.1
✗	✓	✗	✗	✗	✓	✗	BS → HF	88.5	88.8	11.5	88.6	92.9	93.9	7.1	93.4
							HF → BS	91.6	93.5	8.6	92.5	97.4	95.9	2.5	96.6
							TB → BT	85.6	84.4	14.2	85.0	85.8	84.3	14.0	85.0
							BT → TB	63.8	68.2	38.7	65.9	73.2	75.5	27.7	74.3
✗	✓	✗	✗	✗	✗	✓	BS → HF	85.8	87	14.4	86.4	92.7	94.3	7.5	93.5
							HF → BS	96.1	97.9	3.9	97.0	97.6	97.2	2.4	97.4
							TB → BT	88.8	86.9	10.9	87.8	91.8	91.5	8.1	91.6
							BT → TB	78.7	76.9	20.8	77.8	80.0	82.2	20.5	81.1

when the K value was increased to 20, FSLog countered the MITNet+ArcFace combination. In addition, when AMCS and MITNet are used as a single component, the detection performance is significantly improved either in combination with existing modules such as ConvNet or ArcFace, which potentially demonstrates the generalization of the proposed components.

VI. DISCUSSION

A. Why Does FSLog Work?

There are two main reasons why FSLog outperforms state-of-the-art methods in cross-system log anomaly detection. First, recognizing that fuzzy classification boundaries are beneficial for the generalization of novel system log data, we design an innovative adversarial margin cosine loss function to enhance the discriminative and generalization performance of the detection model. Second, noting that existing feature extractors are not robust enough and difficult to represent temporal relationships when facing log data effectively, we design a novel feature extraction module that can simultaneously achieve temporal relationship extraction, anti-noise interference, receptive field expansion, and multi-scale feature fusion.

Notably, meta-learning methods have also been used to address the generalizability problem of log anomaly detection, but we are motivated to use fine-tuning-based few-shot learning and achieve leading detection performance for the following reasons: First, fine-tuning-based few-shot learning utilizes sufficient data from the source domain to adequately learn generalized log patterns and establish strong representation ability. However,

meta-learning (e.g., MetaLog and MAML) relies on task generalization capabilities and needs to simulate multidomain task distributions during the training phase, but cross-domain differences in logs are difficult to cover by limited meta-tasks. Second, fine-tuning-based few-shot learning adjusts only the lightweight classifiers by freezing the backbone network parameters without having to relearn the global features, whereas meta-learning requires adjusting all the parameters in the stage of adapting to the target-domain data, which is susceptible to log noise.

Our study demonstrates the effectiveness of FSLog in cross-system log anomaly detection, though limitations exist. First, when high log parsing error rates cause significant information loss, the MITNet event masking mechanism struggles to extract features from severely noisy sequences. Second, minimal semantic overlap between target and source system logs hinders knowledge transfer from source to target systems. Furthermore, FSLog is more suitable for cold-start phases with scarce target samples, while its accuracy remains insufficient for large-scale data. Given unsupervised models inherent limitations in precision and robustness, O&M personnel can use FSLog to leverage source system prior knowledge for initial training with minimal target system labeled data.

B. Threats to Validity.

Data quality: Although the HDFS, BGL, and Thunderbird datasets have been widely used and publicly available in past studies, they all originate from before 2007 and may not cover the latest features and patterns of current syslogs. Therefore,

conclusions relying on these datasets may be biased and their general applicability in real-world environments cannot be fully guaranteed.

Tool comparison: We implemented log anomaly detection methods including DeepLog, LogAnomaly, PLELog and LogRobust based on literature [10]. For LogTransfer and NeuralLog, we used their public source code for implementation. For LogGroup, MetaLog, and Log-MatchNet, we reproduced them based on the original papers and confirmed that the results are highly consistent with the reported values. Meanwhile, the few-shot learning method uses the open source code from the literature [22]. However, there may be errors or inconsistencies in the conversion process, which could potentially affect the fairness and accuracy of the tool comparisons.

VII. CONCLUSION

In this paper, we have presented FSLog, a method based on the fine-tuning framework to implement cross-system few-shot log anomaly detection. FSLog adopts cosine softmax with adaptive re-scaling factors to train the feature extractor MITNet and pursues the balance between feature transferability and distinguishability by introducing an adversarial margin. To evaluate FSLog, we constructed four datasets based on public data sources. Experimental results show that our approach achieves a significant advantage in initialization scenarios with scarce labels.

Our research still has limitations, as compared to supervised learning models based on sufficient label training, few-shot models exhibit a relatively higher false positive rate. Nonetheless, the application of FSLog in the cold start scenario can effectively avoid the problems of untrainable and overfitting caused by a lack of samples compared to current methods. In our future work, we will further explore how to achieve online deployment of few-shot models.

REFERENCES

- [1] S. He, P. He, Z. Chen, T. Yang, Y. Su, and M. R. Lyu, "A survey on automated log analysis for reliability engineering," *ACM Comput. Surv.*, vol. 54, no. 6, Jul. 2021, Art. no. 130, doi: [10.1145/3460345](https://doi.org/10.1145/3460345).
- [2] C. Zhang, T. Jia, G. Shen, P. Zhu, and Y. Li, "MetaLog: Generalizable cross-system anomaly detection from logs with meta-learning," in *Proc. IEEE/ACM 46th Int. Conf. Softw. Eng.*, 2024, pp. 1899–1910.
- [3] J. Tian, M. Li, L. Chen, Z. Wang, X. Nie, and J. Qin, "SSDALog: Semi-supervised domain adaptation for incremental log-based anomaly detection," *IEEE Trans. Inf. Forensics Secur.*, vol. 20, pp. 6607–6619, 2025.
- [4] M. Du, F. Li, G. Zheng, and V. Srikumar, "DeepLog: Anomaly detection and diagnosis from system logs through deep learning," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, New York, NY, USA, 2017, pp. 1285–1298, doi: [10.1145/3133956.3134015](https://doi.org/10.1145/3133956.3134015).
- [5] W. Meng et al., "LogAnomaly: Unsupervised detection of sequential and quantitative anomalies in unstructured logs," in *Proc. 28th Int. Joint Conf. Artif. Intell.*, 2019, pp. 4739–4745.
- [6] X. Zhang et al., "Robust log-based anomaly detection on unstable log data," in *Proc. 27th ACM Joint Meeting Eur. Softw. Eng. Conf. Symp. Foundations Softw. Eng.*, 2019, pp. 807–817.
- [7] Z. Wang, J. Tian, H. Fang, L. Chen, and J. Qin, "LightLog: A lightweight temporal convolutional network for log anomaly detection on the edge," *Comput. Netw.*, vol. 203, 2022, Art. no. 108616. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1389128621005119>
- [8] L. Yang et al., "Semi-supervised log-based anomaly detection via probabilistic label estimation," in *Proc. IEEE/ACM 43rd Int. Conf. Softw. Eng.*, 2021, pp. 1448–1460.
- [9] R. Chen et al., "LogTransfer: Cross-system log anomaly detection for software systems with transfer learning," in *Proc. IEEE 31st Int. Symp. Softw. Rel. Eng.*, 2020, pp. 37–47.
- [10] V.-H. Le and H. Zhang, "Log-based anomaly detection with deep learning: How far are we?," in *Proc. 44th Int. Conf. Softw. Eng.*, 2022, pp. 1356–1367, doi: [10.1145/3510003.3510155](https://doi.org/10.1145/3510003.3510155).
- [11] C. Han, B. Guan, T. Li, D. Kang, J. Qin, and Y. Wu, "Few-shot log anomaly detection based on matching networks," *IEEE Trans. Netw. Service Manag.*, vol. 21, no. 3, pp. 2909–2925, Jun. 2024.
- [12] C. Xu, J. Shen, and X. Du, "A method of few-shot network intrusion detection based on meta-learning framework," *IEEE Trans. Inf. Forensics Secur.*, vol. 15, pp. 3540–3552, 2020.
- [13] J. Li, H. He, S. Chen, D. Jin, and J. Yang, "LogGraph: Log event graph learning aided robust fine-grained anomaly diagnosis," *IEEE Trans. Dependable Secure Comput.*, vol. 21, no. 4, pp. 1876–1889, Jul./Aug., 2024.
- [14] J.-G. Lou, Q. Fu, S. Yang, Y. Xu, and J. Li, "Mining invariants from console logs for system problem detection," in *Proc. USENIX Conf. USENIX Annu. Tech. Conf.*, 2010, pp. 1–14.
- [15] W. Xu, L. Huang, A. Fox, D. Patterson, and M. I. Jordan, "Detecting large-scale system problems by mining console logs," in *Proc. ACM SIGOPS 22nd Symp. Operating Syst. Princ.*, New York, NY, USA, 2009, pp. 117–132.
- [16] H. Studiawan, F. Sohel, and C. Payne, "Anomaly detection in operating system logs with deep learning-based sentiment analysis," *IEEE Trans. Dependable Secure Comput.*, vol. 18, no. 5, pp. 2136–2148, Sep./Oct. 2021.
- [17] V.-H. Le and H. Zhang, "Log-based anomaly detection without log parsing," in *Proc. 36th IEEE/ACM Int. Conf. Automated Softw. Eng.*, 2022, pp. 492–504.
- [18] X. Li, P. Chen, L. Jing, Z. He, and G. Yu, "SwissLog: Robust anomaly detection and localization for interleaved unstructured logs," *IEEE Trans. Dependable Secure Comput.*, vol. 20, no. 4, pp. 2762–2780, Jul./Aug. 2023.
- [19] J. Tian, M. Li, Z. Wang, L. Chen, J. Qin, and R. Zhang, "OMLog: Online log anomaly detection for evolving system with meta-learning," *IEEE Internet Things J.*, vol. 12, no. 15, pp. 30142–30155, Aug. 2025.
- [20] S. Lu, X. Wei, Y. Li, and L. Wang, "Detecting anomaly in Big Data system logs using convolutional neural network," in *Proc. IEEE 16th Int. Conf. Dependable, Autonomic Secure Comput., 16th Int. Conf. Pervasive Intell. Comput., 4th Int. Conf. Big Data Intell. Comput. Cyber Sci. Technol. Congr.*, 2018, pp. 151–158.
- [21] W. Meng et al., "A semantic-aware representation framework for online log analysis," in *Proc. 29th Int. Conf. Comput. Commun. Netw.*, 2020, pp. 1–7.
- [22] W. Li et al., "LibFewShot: A comprehensive library for few-shot learning," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 12, no. 12, pp. 14938–14955, Dec. 2023.
- [23] W.-Y. Chen, Y.-C. Liu, Z. Kira, Y.-C. F. Wang, and J.-B. Huang, "A closer look at few-shot classification," in *Proc. Int. Conf. Learn. Representations*, 2019, pp. 1–16.
- [24] K. Zheng, H. Zhang, and W. Huang, "DiffKendall: A novel approach for few-shot learning with differentiable Kendall's rank correlation," in *Proc. 37th Int. Conf. Neural Inf. Process. Syst.*, Red Hook, NY, USA: Curran Associates Inc., 2024, pp. 49403–49415.
- [25] C. Finn, P. Abbeel, and S. Levine, "Model-agnostic meta-learning for fast adaptation of deep networks," in *Proc. 34th Int. Conf. Mach. Learn.*, 2017, pp. 1126–1135.
- [26] J. Oh, H. Yoo, C. Kim, and S. Yun, "BOIL: Towards representation change for few-shot learning," in *Proc. Int. Conf. Learn. Representations*, 2021, pp. 1–24.
- [27] J. Snell, K. Swersky, and R. Zemel, "Prototypical networks for few-shot learning," in *Proc. 31st Int. Conf. Neural Inf. Process. Syst.*, Red Hook, NY, USA: Curran Associates Inc., 2017, pp. 4080–4090.
- [28] J. Xie, F. Long, J. Lv, Q. Wang, and P. Li, "Joint distribution matters: Deep Brownian distance covariance for few-shot classification," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 7962–7971.
- [29] J. Deng, J. Guo, and S. Zafeiriou, "ArcFace: Additive angular margin loss for deep face recognition," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2018, pp. 4685–4694.
- [30] B. Liu et al., "Negative margin matters: Understanding margin in few-shot classification," in *Proc. Eur. Conf. Comput. Vis.*, 2020, pp. 438–455.
- [31] M. Kim, A. K. Jain, and X. Liu, "AdaFace: Quality adaptive margin for face recognition," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 18729–18738.

- [32] Y. Wen, K. Zhang, Z. Li, and Y. Qiao, "A discriminative feature learning approach for deep face recognition," in *Proc. Eur. Conf. Comput. Vis.*, 2016, pp. 499–515.
- [33] W. Liu, Y. Wen, Z. Yu, and M. Yang, "Large-margin softmax loss for convolutional neural networks," in *Proc. 33rd Int. Conf. Mach. Learn.*, New York, New York, USA, 2016, pp. 507–516.
- [34] W. Liu, Y. Wen, Z. Yu, M. Li, B. Raj, and L. Song, "SphereFace: Deep hypersphere embedding for face recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2017, pp. 6738–6746.
- [35] H. Wang et al., "CosFace: Large margin cosine loss for deep face recognition," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2018, pp. 5265–5274.
- [36] B. Li, B. Yang, C. Liu, F. Liu, R. Ji, and Q. Ye, "Beyond max-margin: Class margin equilibrium for few-shot object detection," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2021, pp. 7359–7368.
- [37] E. Eldele, M. Ragab, Z. Chen, M. Wu, and X. Li, "TSLANet: Rethinking transformers for time series representation learning," in *Proc. 41st Int. Conf. Mach. Learn.*, 2024, pp. 12409–12428.
- [38] S. Bai, J. Z. Kolter, and V. Koltun, "An empirical evaluation of generic convolutional and recurrent networks for sequence modeling," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2018.
- [39] Z. Yue et al., "TS2Vec: Towards universal representation of time series," in *Proc. AAAI Conf. Artif. Intell.*, 2022, pp. 8980–8987.
- [40] F. Wang and H. Liu, "Understanding the behaviour of contrastive loss," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2021, pp. 2495–2504.
- [41] M. Du and F. Li, "Spell: Streaming parsing of system event logs," in *Proc. IEEE 16th Int. Conf. Data Mining*, 2016, pp. 859–864.
- [42] A. Oliner and J. Stearley, "What supercomputers say: A study of five system logs," in *Proc. 37th Annu. IEEE/IFIP Int. Conf. Dependable Syst. Netw.*, 2007, pp. 575–584.
- [43] Y. Tian, Y. Wang, D. Krishnan, J. B. Tenenbaum, and P. Isola, "Rethinking few-shot image classification: A good embedding is all you need?," in *Proc. 16th Eur. Conf. Comput. Vis. Proc. Eur. Conf. Comput. Vis.*, Glasgow, U.K.: Springer-Verlag, 2020, pp. 266–282.
- [44] L. Bertinetto, J. F. Henriques, P. Torr, and A. Vedaldi, "Meta-learning with differentiable closed-form solvers," in *Proc. Int. Conf. Learn. Representations*, 2019, pp. 1–15. [Online]. Available: <https://openreview.net/forum?id=HyxnZh0ct7>
- [45] F. Sung, Y. Yang, L. Zhang, T. Xiang, P. H. Torr, and T. M. Hospedales, "Learning to compare: Relation network for few-shot learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2018, pp. 1199–1208.
- [46] R. Hou, H. Chang, B. MA, S. Shan, and X. Chen, "Cross attention network for few-shot classification," in *Proc. Adv. Neural Inf. Process. Syst.*, 2019, pp. 4003–4014.
- [47] D. Kang, H. Kwon, J. Min, and M. Cho, "Relational embedding for few-shot classification," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2021, pp. 8802–8813.
- [48] P. Goyal et al., "Accurate, large minibatch SGD: Training imagenet in 1 hour," 2017, *arXiv:1706.02677*.
- [49] K. Lee, S. Maji, A. Ravichandran, and S. Soatto, "Meta-learning with differentiable convex optimization," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 10649–10657.



Jiayu Tian received the master's degree in cyberspace security from Dalian University, Dalian, China, where he is currently working toward the PhD degree. His current research interests include log anomaly detection and cyberspace security.



Mingchu Li received the doctorate degree in mathematics from the University of Toronto, in 1998. He was with the School of Software, Tianjin University, as a full professor from 2002 to 2004, School of Software Technology, Dalian University of Technology, as a full professor and the vice dean from 2002 to 2023. He has been with Jiangxi Normal University as a full professor since 2024. His main research interests include theoretical computer science and information security, trust models, and cooperative game theory.



Jianyuan Gan received the BS and MS degrees in science from Nanchang Hangkong University, Nanchang, China, in 2016 and 2019, respectively. He is currently working toward the EngD degree with the Dalian University of Technology. His research interests include traffic engineering and AI.